
Seals MES 系统

开发环境系统监控指南

文档版本：V1.0

编制日期：2026年5月15日

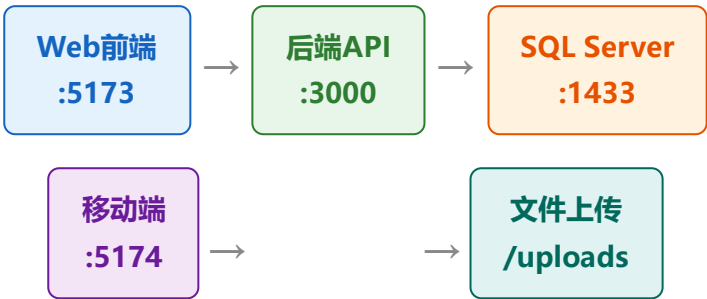
适用环境：Windows 开发环境

目录

1. 系统架构概览
2. 健康检查接口
3. 进程与服务监控
4. 日志系统
5. SSE 实时推送监控
6. 关键配置文件
7. 快速诊断脚本
8. 生产环境监控
9. 常见故障排查
10. 监控最佳实践

1. 系统架构概览

1.1 三端架构



1.2 技术栈监控能力

层次	技术栈	监控能力
前端	Vue 3 + Vite + Ant Design Vue	Vite HMR 热更新、浏览器 DevTools
后端	Express + TypeScript + Sequelize	健康检查、Pino日志、SSE推送、请求追踪
数据库	SQL Server	健康检查连通性、Sequelize查询日志
进程管理	nodemon (dev) / PM2 (prod)	进程状态、内存、自动重启

2. 健康检查接口

2.1 接口定义

项目	说明
路径	GET /api/v1/health
认证	无需认证 (公开端点)
成功状态码	200
降级状态码	503 (数据库异常)

2.2 返回结构

```
{
  "status": "healthy",          // healthy | degraded
  "timestamp": "2026-05-26T15:00:13.000Z",
  "uptime": "2h 35m",          // 服务运行时长
  "database": "ok",             // ok | error
  "memory": {
    "rss": "156MB",             // 进程驻留内存
    "heapUsed": "89MB",         // V8堆已用
    "heapTotal": "128MB"        // V8堆总量
  }
}
```

2.3 实现逻辑

```
// server/src/app.ts
app.get(`${API_PREFIX}/health`, async (_req, res) => {
  const uptime = process.uptime();
  const memUsage = process.memoryUsage();
  let dbStatus = 'ok';
  try {
    await sequelize.authenticate(); // 测试数据库连接
  } catch {
    dbStatus = 'error';
  }
  res.status(dbStatus === 'ok' ? 200 : 503).json({
    status: dbStatus === 'ok' ? 'healthy' : 'degraded',
    timestamp: new Date().toISOString(),
    uptime: `${Math.floor(uptime / 3600)}h ${Math.floor((uptime % 3600) / 60)}m`,
    database: dbStatus,
    memory: {
      rss: `${Math.round(memUsage.rss / 1024 / 1024)}MB`,
      heapUsed: `${Math.round(memUsage.heapUsed / 1024 / 1024)}MB`,
      heapTotal: `${Math.round(memUsage.heapTotal / 1024 / 1024)}MB`
    }
  });
});
```

2.4 字段说明

字段	类型	含义	异常阈值
status	string	服务整体状态	degraded 表示数据库异常
uptime	string	进程运行时长	频繁重置需排查
database	string	数据库连通性	error 表示连接失败

memory.rss	string	进程驻留内存	超过 800MB 需关注
memory.heapUsed	string	V8堆已用内存	接近 heapTotal 需关注

3. 进程与服务监控

3.1 端口检查

```
# 检查后端服务（端口 3000）
netstat -ano | findstr :3000

# 检查 Web 前端（端口 5173）
netstat -ano | findstr :5173

# 检查移动端（端口 5174）
netstat -ano | findstr :5174
```

3.2 进程管理

操作	命令	说明
查看端口占用	netstat -ano findstr :3000	获取 PID
杀死进程	Stop-Process -Id <PID> -Force	PowerShell 命令
启动后端	cd server; npm run dev	nodemon 热重载
启动前端	cd client; npm run dev	Vite 开发服务器
启动移动端	cd client-mobile; npm run dev	Vite 开发服务器

PowerShell 注意事项： PowerShell 不支持 && 作为命令分隔符，请使用分号；代替。

3.3 nodemon 开发模式

后端使用 nodemon 监听文件变更，自动重启服务。关键行为：

事件	行为
.ts 文件修改保存	自动重启服务
重启中收到请求	请求排队等待
手动重启	终端输入 rs 回车
忽略目录	dist/ 目录变更不触发重启

4. 日志系统

4.1 Pino 结构化日志

系统采用 Pino 作为日志框架，提供结构化 JSON 输出、日志级别控制、请求追踪和模块标识。

4.1.1 配置文件

```
// server/src/config/logger.ts
const logger = pino({
  level: process.env.LOG_LEVEL || (isDev ? 'debug' : 'info'),
  transport: isDev
    ? { target: 'pino-pretty', options: { colorize: true, translateTime: 'SYS:yyyy-mm-dd HH:MM:ss.l' } }
    : undefined,
  base: { service: 'seals-mes', env: process.env.NODE_ENV || 'development' },
  timestamp: pino.stdTimeFunctions.isoTime,
});
```

4.1.2 日志级别

级别	用途	开发环境	生产环境
debug	调试信息、变量值	✓ 输出	✗ 不输出
info	常规操作日志	✓ 输出	✓ 输出
warn	警告、4xx错误	✓ 输出	✓ 输出
error	错误、5xx异常	✓ 输出	✓ 输出
fatal	致命错误	✓ 输出	✓ 输出

4.1.3 日志级别控制

```
# 在 server/.env 中设置（降低日志噪音）
LOG_LEVEL=info

# 可选值：debug, info, warn, error, fatal
# 不设置时：开发环境默认 debug，生产环境默认 info
```

4.2 模块日志标识

通过 createLogger(module) 创建带模块标识的子日志器，方便按模块过滤：

模块标识	关注内容	文件位置
http	每个HTTP请求的方法/路径/状态码/耗时	requestLogger.middleware.ts
finishedGoods	出入库操作、库存变动、批次生成	finishedGoods.ts
approval	审批流执行、状态流转	approval.service.ts
subscriber-taskCompletion	报工→工序状态同步	taskCompletion.subscriber.ts
subscriber-lineside	线边仓流转	linesideMovement.subscriber.ts
stockCountScheduler	自动盘点定时任务	stockCountScheduler.service.ts
eventWorker	事件工作器循环	eventWorker.service.ts
workReport-autoInbound	报工自动入库	autoInbound.ts
workReport-taskSync	报工→任务同步	taskSync.ts
salesOrderSync	销售订单状态同步	salesOrderSync.service.ts
backflushTask	倒冲任务执行	backflushTask.service.ts
workflow	工作流引擎	workflow.engine.ts
productionYield	生产产出计算	productionYield.service.ts

4.3 HTTP 请求日志

每个 HTTP 请求自动记录完整生命周期：

```
// 请求开始
[INFO] http: GET /api/v1/return-orders { requestId: "uuid-xxx", method: "GET", path: "/api/v1/return-orders", userId: 1 }

// 请求完成（含耗时）
[INFO] http: GET /api/v1/return-orders 200 45ms { requestId: "uuid-xxx", statusCode: 200, duration: 45 }

// 4xx 警告
[WARN] http: POST /api/v1/return-orders/RT-xxx/cancel 400 12ms { statusCode: 400, duration: 12 }
```



```
// 5xx 错误
[ERROR] http: GET /api/v1/return-orders 500 2300ms { statusCode: 500, duration: 2300 }
```

性能基线：正常 API 响应 < 200ms；200ms~1s 需关注慢查询；> 1s 通常为数据库性能问题。

4.4 日志输出格式

环境	输出格式	示例
开发环境	pino-pretty 美化输出（彩色、时间格式化）	[2026-05-26 15:00:13.534] INFO: 工序状态变更订阅者已注册
生产环境	JSON 格式（便于 ELK 采集）	{"level":"info","time":"2026-05-26T07:00:13.534Z","msg":"工序状态变更订阅者已注册"}

5. SSE 实时推送监控

5.1 架构说明

系统内置 SSE (Server-Sent Events) 推送服务，用于移动端实时接收通知。每个用户连接维护一个 Response 对象，心跳保活每30秒发送注释行，连接断开时自动清理。

5.2 事件类型

事件类型	说明	触发场景
notification	系统通知	消息推送
approval	审批通知	审批流状态变更
task_update	任务更新	生产任务状态变更
system	系统消息	全局广播

5.3 心跳机制

```
// 每 30 秒发送心跳
:heartbeat      // SSE 注释行，客户端自动忽略

// 客户端通过心跳判断连接存活
// 超过 60 秒未收到心跳或数据 → 判定连接断开 → 自动重连
```

6. 关键配置文件

6.1 环境变量 (.env)

```
# server/.env
PORT=3000                # 后端服务端口
DB_HOST=127.0.0.1        # 数据库主机
DB_PORT=1433             # 数据库端口
DB_NAME=SEALSMES         # 数据库名称
DB_USER=sa               # 数据库用户
DB_PASSWORD=*****       # 数据库密码
JWT_SECRET=*****        # JWT签名密钥
JWT_EXPIRES_IN=24h        # Token有效期
JWT_REFRESH_EXPIRES_IN=7d # 刷新Token有效期
```

UPLOAD_DIR=uploads	# 上传目录
MAX_FILE_SIZE=10485760	# 最大文件大小(10MB)
LOG_LEVEL=debug	# 日志级别(可选, 不设置则默认)

6.2 PM2 生产部署配置

```
// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'seals-mes-server',
    script: 'src/server.ts',
    interpreter: 'npx',
    interpreter_args: 'ts-node -r tsconfig-paths/register',
    cwd: './server',
    instances: 1, // Windows: fork模式单进程
    exec_mode: 'fork', // Linux生产: 改为 cluster + instances: 'max'
    max_memory_restart: '1G', // 内存超1G自动重启
    env_production: {
      NODE_ENV: 'production',
      PORT: 3000
    }
  ]
};
```

6.3 Vite 开发代理

```
// client/vite.config.ts
server: {
  proxy: {
    '/api/v1': {
      target: 'http://localhost:3000', // 后端地址
      changeOrigin: true
    },
    '/uploads': {
      target: 'http://localhost:3000',
      changeOrigin: true
    }
  }
}
```

重要: Vite 仅代理 `/api/v1` 和 `/uploads` 前缀的请求。如果 API 路径写错 (如 `/api/` 而非 `/api/v1/`) , 请求会被 SPA fallback 拦截, 返回前端 `index.html` 而非 API 响应。

7. 快速诊断脚本

7.1 健康检查

```
# PowerShell 一键检查
Invoke-RestMethod http://localhost:3000/api/v1/health

# 仅检查数据库连通性
(Invoke-RestMethod http://localhost:3000/api/v1/health).database

# 仅查看内存
(Invoke-RestMethod http://localhost:3000/api/v1/health).memory
```

7.2 端口状态快速检查

```
# 检查三端服务是否运行
$services = @( @{Name="后端"; Port=3000}, @{Name="Web前端"; Port=5173}, @{Name="移动端"; Port=5174} )
foreach ($s in $services) {
    $result = netstat -ano | findstr ":$($s.Port) " | findstr "LISTENING"
    if ($result) { Write-Host "$($s.Name) ($($s.Port)): ✔ 运行中" -ForegroundColor Green }
    else { Write-Host "$($s.Name) ($($s.Port)): ✘ 未启动" -ForegroundColor Red }
}
```

7.3 请求耗时测试

```
# 测量API响应时间
Measure-Command { Invoke-RestMethod http://localhost:3000/api/v1/health } | Select-Object TotalMilliseconds
```

8. 生产环境监控

8.1 PM2 常用命令

命令	用途
pm2 list	查看进程状态、重启次数、CPU/内存占用

pm2 monit	实时监控面板 (CPU/内存曲线)
pm2 logs	实时日志流 (Ctrl+C 退出)
pm2 logs seals-mes-server --lines 100	查看最近100行日志
pm2 restart seals-mes-server	重启服务
pm2 stop seals-mes-server	停止服务
pm2 describe seals-mes-server	查看进程详细信息

8.2 NSSM Windows 服务方式

如需将后端注册为 Windows 服务 (开机自启) , 可使用 NSSM:

```
# 安装服务
nssm install seals-mes "C:\Program Files\nodejs\node.exe" "D:\rubber\Seals MES System\server\src\server.ts"

# 配置自动重启
nssm set seals-mes AppParameters "--require tsconfig-paths/register src/server.ts"
nssm set seals-mes AppDirectory "D:\rubber\Seals MES System\server"
nssm set seals-mes StartServiceDelay 5000

# 启动/停止
nssm start seals-mes
nssm stop seals-mes
```

8.3 内存监控阈值

指标	正常范围	告警阈值	危险阈值
rss	< 300MB	300~500MB	> 500MB
heapUsed	< 200MB	200~400MB	> 400MB
heapTotal	< 300MB	300~500MB	> 500MB
uptime	持续增长	频繁重置	每分钟重启

PM2 配置了 `max_memory_restart: '1G'` , 进程内存超过 1GB 时自动重启。

9. 常见故障排查

#	症状	可能原因	排查步骤
1	API 请求返回管理驾驶舱页面	Vite 代理未匹配 API 路径	① 检查 URL 是否以 /api/v1/ 开头 ② 检查 vite.config.ts 代理规则
2	401 未提供认证令牌	新标签页不携带 Authorization header	① 检查打印URL是否携带 ?token=xxx ② 检查 auth.middleware.ts 是否支持 query token
3	端口 3000 被占用	旧进程未释放	① netstat -ano findstr :3000 ② Stop-Process -Id <PID> -Force ③ 重新启动
4	数据库连接失败	SQL Server 未启动或 .env 配置错误	① 检查 SQL Server 服务状态 ② 验证 .env 中 DB_HOST/DB_PORT/DB_NAME ③ Invoke-RestMethod /api/v1/health 查看 database 字段
5	API 响应慢 (> 1s)	数据库慢查询	① 查看请求日志中的 duration ② 检查 SQL Server 活动监视器 ③ 考虑添加索引
6	前端修改不生效	Vite HMR 未触发 / 浏览器缓存	① 强制刷新 (Ctrl+Shift+R) ② 检查终端 Vite 是否检测到文件变更
7	后端修改不生效	nodemon 未检测到变更	① 检查终端 nodemon 是否重启 ② 手动输入 rs 触发重启
8	内存持续增长	内存泄漏	① 定期检查 /api/v1/health 内存字段 ② 使用 Node.js --inspect 调试 ③ PM2 自动重启兜底

10. 监控最佳实践

10.1 开发环境日常检查清单

#	检查项	频率	方法
---	-----	----	----

1	三端服务运行状态	每日开始开发前	端口检查脚本
2	后端健康检查	功能测试前	Invoke-RestMethod /api/v1/health
3	后端日志异常	持续	终端观察 ERROR/WARN 级别
4	API 响应时间	功能测试时	请求日志 duration 字段
5	数据库连接状态	数据库操作异常时	health 接口 database 字段

10.2 日志使用规范

推荐做法：

- 业务关键操作使用 `log.info()`（入库、出库、审批、确认等）
- 可恢复异常使用 `log.warn()`（入库失败但不影响确认）
- 不可恢复错误使用 `log.error()`（数据库异常、事务回滚）
- 调试信息使用 `log.debug()`（变量值、中间计算结果）
- 每条日志携带上下文对象：`log.info({ orderNo }, '订单创建成功')`

避免做法：

- 禁止使用 `console.log()`，统一使用 Pino 日志器
- 禁止在日志中输出敏感信息（密码、Token、个人信息）
- 禁止在生产环境使用 `LOG_LEVEL=debug`

10.3 告警建议

告警项	条件	级别	通知方式
服务宕机	health 接口连续 3 次超时	P0-紧急	即时通知
数据库异常	health.database = 'error'	P0-紧急	即时通知
内存泄漏	rss 连续 5 分钟 > 500MB	P1-重要	5分钟内通知
频繁重启	PM2 restarts > 3次/小时	P1-重要	5分钟内通知
慢请求	duration > 3s	P2-一般	日志记录
5xx 错误	statusCode >= 500	P1-重要	5分钟内通知